

Vue.js – Övningar

Vue Dag 1 (övning 1 till 10)

Kodexempel finns att utgå ifrån i vissa av övningarna (vue-dag1.zip)

Övning 1

Skapa en första html-sida där du använder Vue genom:

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

Skapa en instans av Vue kalla den "app". Koppla den till ett element i HTML. Skriv ut ett meddelande genom Vue till elementet du har kopplat det till.

Se 1-vue-dag1-introduktion.html

Övning 2

Utveckla övning 1 genom att låta användaren ändra texten i ett input-fält.

Se 2-vue-dag1-introduktion-vmodel.html

Övning 3

Skriv en hälsning till användaren beroende på vilken tid det är på dygnet. T ex. "God förmiddag", "God middag".

Se 8-vue-dag1-events-3.html för att se hur man använder inmatning och en metod. Lägg koden för att skapa hälsningen i en metod (på liknande sätt som exemplet).

I Vue skriver man Javascript som vanligt så det vi tidigare har lärt oss kan vi använda för att skriva koden för att göra hälsningen dvs deklarerar variabler med let, if-satser mm.

Tänk på: Man måste använda this i en metod för att ändra värdet på en variabel som är deklarerad i

```
data() {  
  return {  
    message: 'Hello Vue',  
  }  
}
```

t ex i metoden: this.message = 'Ny text'

Se vue-dag1-ovn3-kodexempel.html i vue-ovningar-kodexempel.zip för lösningsförslag.

Övning 4 – Konvertera Celcius till Fahrenheit

Man ska kunna konvertera Celsius till Fahrenheit.

$$\text{fahrenheit} = \text{celsius} * 9 / 5 + 32;$$

Se 8-vue-dag1-events-3.html för att se hur man använder inmatning och en metod. Lägg koden för att göra temperatur-omvandlingen i en metod (på liknande sätt som exemplet).

Övning 5 - Handlingslista

Gör ett enkelt program som har en array med t ex inköpslista. Man ska kunna visa listan och användaren ska även kunna lägga till en ny vara som ska sparas i arrayen och visas i listan.

Utgå från 4-vue-dag1-lists-add.html (Här är det en att-göra-lista)

Övning 6 - Fortsättning Handlingslista

Bygg ut programmet i övning 5 så att man kan slumpa fram en vara ur listan genom att användaren trycker på en knapp med `v:onclick (@click)`.

Använd `Math.random` i Javascript för slump:

```
let randTodo = Math.floor(Math.random()*5);  
(let randTodo = Math.floor(Math.random()*this.todos.length());)
```

Se `vue-dag1-ovn6-kodexempel.html` i `vue-ovningar-kodexempel.zip` för lösningsförslag.

Övning 7 - Bindings

Gör en sida som har ett html-element t ex en p-tagga som du formaterar med Class Binding (`:class`) t ex ändra färg, font-storlek, bakgrundsfärg. Klassen som du använder måste finnas i css-filen.

Se exempel 11-vue-dag1-bindings-class-1.html för exempel. (Här använder vi klassen `critical`)

Övning 8 – Bindings och directives

Nu ska vi göra en sida där man kan använda *Class Bindings* som i övning 7 men nu ska vi låta användaren påverka ett html-elements formatering.

Se 12-vue-dag1-bindings-class-2.html för exempel.

Du kan utgå från koden i övning 7.

Lägg till så att det finns en knapp, trycker man på den ändras formateringen på ett html-element t ex en p-tagga.

Övning 9

Gör gärna den här övningen som är lite tillsammans med någon.

Gör ett enkelt Gissa-spel där användaren får gissa ett nummer mellan 1 och 10. Du slumpar fram ett nummer mellan 1 och 10 och ger användaren ledtrådar om hen gissat för lågt eller högt.

Se 8-vue-dag1-events-3.html för att se hur man använder inmatning och en metod.

Tips på vägen: Du behöver tre variabler: meddelande, slumptal, användarens gissning som behöver ligga i/deklareras i data-blocket. Själva koden för gissningen lägger du i en metod. Och på samma sätt som i övning 3 och 4 skriver du Javascript-kod som vanligt för att lösa själva koden för spelet.

Nedan är ett förslag på pseudokod.

Lösningförslag finns i [vue-ovningar-kodexempel.zip](#) [vue-dag1-ovn9-kodexempel.html](#)

/*

Användaren ska gissa ett nummer mellan 1 och 10

Datorn slumpar fram ett nummer som användarens gissning jämförs med

Om numret är för lågt får man ett meddelande "Ditt nummer är för lågt!!

Om numret är för högt får man ett meddelande "Ditt nummer är för högt!"

Gissar man rätt får man meddelandet "Du har gissat rätt!"

Vi ska använda slump med Math.random

Vi ska visa numret användaren har gissat

Vi ska visa meddelande beroende på "resultatet" på gissningen

Gissar man rätt måste det skapas ett nytt slumptal

*/

Övning 10 – Fortsättning på Gissaspelet (med Directives v-if)

Lite svårare!

Nu ska vi bygga ut Gissaspelet (övning 9) så att det finns en knapp för att göra ett nytt spel när en omgång är klar dvs när man har gissat rätt.

När man har gissat rätt ska formuläret för att gissa dvs ett textfält och en knapp döljas och bara knappen för nytt spel visas.

När man trycker på knappen för ett nytt spel visas formuläret igen och knappen för nytt spel döljs igen.

Ett tips är att ha en variabel som ändras beroende på om ett spel/omgång är avslutat eller inte t ex `isFinishedGame` som sedan används med `v-if` i html-koden och på sätt bestämmer vad som ska visas eller inte.

Se `10-vue-dag1-directive-vif-html` för en ledtråd.

Lösningförslag finns i `vue-ovningar-kodexempel.zip` `vue-dag2-ovn10-kodexempel.html`

Vue Dag 2 (övning 11 till 14)

Kodexempel finns att utgå ifrån i vissa av övningarna (`vue-dag2.zip`)

Övning 11 – Ett lite större program (Handlingslista) Del 1

Utgå från `1-vue-dag2-lists-forts-1.html`

Här använder vi ett objekt som sparas i en array och inte en array som sparar strängar som i exemplet `vue-introduktion-list-2.html` och övning 5 och 6.

Du ska göra om kodexemplet så att det blir en handlingslista (nu är det en att-göra-lista).

Övning 12 – Ett lite större program (Handlingslista) Del 2

Utveckla programmet så att man kan lägga till antal för varje vara man ska handla. Lägg också till att man kan radera en vara.

Fördjupning: Du ska också lägga till en knapp för att markera om varan är köpt. Ändra färgen på varan i listan när man trycker på knappen så att man ser att den är markerad som köpt. Titta på exemplet `vue-introduktion-bindings-directive.html` och övning 7 och 8.

Exempel finns i `2-vue-dag2-lists-forts-2.html` att utgå ifrån.

Övning 13 - Components

Gör en component som heter `button-counter` som räknar antal klick användaren gör när hen klickar på en knapp.

Du kan titta på kodexemplet `3-vue-dag2-components-1.html`

Du kan också lägga den delen av koden som räknar antal klick i en metod som tillhör din component. Titta på exemplet `4-vue-dag2-components-2.html`

Övning 14 - Components med props

Utgå från exemplet `6-vue-dag2-components-props-2.html` som har delat upp koden så att det finns en array som innehåller ett objekt som man kan komma åt genom en props i en component.

Gör om kodexemplet så att det istället innehåller en handlingslista. (Det svåra kan vara att hålla isär vad som är en props och vad som är själva arrayen, som ligger utanför din component).

Vue Dag 3, Vue Vite och Router (övning 15 till 21)

Övning 15 - Skapa ett projekt med Vue Vite

Skapa ett projekt med Vue Vite (som default använder det Composition API).

```
npm create vue@latest
```

Följ instruktionerna:

1. Namnge projektet
2. Välj vilka tillägg som ska vara med t.ex Router
3. Gå till projektmappen, installera dependencies, starta dev-server:

```
cd ditt-projektnamn
```

```
npm install
```

```
npm run dev
```

Övning 16 - Skapa en ny component

1. Skapa en ny component som heter `ClickCount.vue` i `components`-mappen.
2. Den ska räkna antal klick när man trycker på en knapp.
3. Importera och använd `ClickCount` i `App.vue`.

Du behöver ha en button med on-click (@click) som anropar en function som räknar upp antal klick som i en variabel genom ref()

```
import { ref } from 'vue'
```

```
const count = ref(0) "Här har count initialt värdet 0"
```

Tänk på att med Composition API behöver man använda .value för att komma åt värdet för variabeln.

Övning 17 - Utveckla din ClickCount component

Du ska räkna antal klick på samma sätt som i övning 16 men inaktivera knappen efter 5 klick.

Övning 18 - Skapa en component som innehåller ett formulär

1. Skapa en ny component som heter `Greeting.vue` i `components`-mappen.
2. Du ska ha ett formulär där du matar in ditt namn och sedan visar en hälsning tillsammans med namnet. Återställ input-fältet så det är tomt efter du har gjort en submit och visat hälsningen.

3. Importera och använd `Greeting` i `App.vue`.

Övning 19 - Router

Skapa ett projekt med Vue Vite (som default använder det Composition API).

Se till att välja Router när du skapar projektet.

```
npm create vue@latest
```

Följ instruktionerna:

1. Namnge projektet
2. Välj vilka tillägg som ska vara med t.ex Router
3. Gå till projektmappen, installera dependencies, starta dev-server:

```
cd ditt-projektnamn
```

```
npm install
```

```
npm run dev
```

```
--
```

1. Skapa en ny view som heter `Contact`.
2. Gå till router/index.js och se till att din view `Contact` finns med för routing.
3. Lägg sedan till en länk till din view i App.vue genom router-link

Övning 20 - Gör om Gissaspelet från övning 9 och 10

Du ska göra om Gissaspelet så att du använder Vue vite med Composition API och Router.

Gör gärna denna övningen tillsammans med någon och gå genom/förklara för varandra vad ni ska tänka på till skillnad från det vi har gått genom tidigare nu när vi använder composition API.

Applikationen ska fungera på samma sätt men tänk på:

Nu har vi inte methods-blocket utan skriver function direkt i koden. Vi använder ref för variabler.

Skapa också en FAQ-sida med lite information om spelet. Titta på övning 19 hur du har gjort med Router.

OBS! Tänk på att med Composition API behöver man använda .value för att komma åt värdet för en variabel deklarerad med ref().

Övning 21 - Gör om Handlingslistan från övning 11 och 12

Du ska göra om Handlingslistan så att du använder Vue vite med Composition API.

Tänk på samma sätt som i övning 20 när du skriver om koden.